Perfect — you're ready to evolve your **hybrid matching engine** into a true **two-way search system** (startups → investors *and* investors → startups).

Let's break this into a **mindmap-style overview** first, so you can see exactly how everything connects before touching the code.

🧠 Two-Way Matching Engine — Mindmap of the Full Flow

```
HYBRID MATCHING ENGINE
│
├── 🏁 Initialization
│   ├─ Load environment variables (.env)
│   ├─ Connect to Elasticsearch (local or cloud)
│   ├─ Load SentenceTransformer model
│   └─ Define constants (INDEX_NAME, thresholds, etc.)
│
├── 📦 Data Ingestion
│   ├─ Load Startups from startups.csv → List[Startup]
│   └─ Load Investors from investors.csv → List[Investor]
│
├── 🔢 Embedding & Indexing
│   ├─ For INVESTOR side:
│   │   ├─ Compute embedding from investor.description
│   │   ├─ Store document in ES index "investors"
│   │   └─ description_vector (dense_vector)
│   │
│   └─ For STARTUP side (NEW):
│       ├─ Compute embedding from startup.description
│       ├─ Store document in ES index "startups"
│       └─ description_vector (dense_vector)
│
├── ⚙️ Matching Logic
│   ├─ Rule-based Score (explicit data)
│   │   ├─ Industry overlap
│   │   ├─ Stage alignment
│   │   ├─ Funding goal within investor check range
│   │   ├─ Location, team size, revenue stage
│   │   └─ Portfolio & experience bonus
│   │
│   ├─ Semantic Score (embedding similarity)
│   │   ├─ For startups → investors (current)
│   │   └─ For investors → startups (NEW reverse direction)
│   │
│   ├─ Combine into Final Score:
│   │     final = 0.7 * rule + 0.3 * semantic
│   │
│   └─ Generate LLM Justification if above threshold
│
├── 🔍 Lookup Interfaces
│   ├─ Startup → Investor (current)
```

```
│    ├─ Investor → Startup (current)
│    └─ Both → Cross-Query (two-way hybrid)
│        ├─ Startup to find matching investors (semantic + rules)
│        └─ Investor to find startups (semantic + rules)
│
├── 💾 Output
│    ├─ Top matches (printed + JSON file)
│    ├─ Justifications (LLM text)
│    └─ Option to re-run or view by investor/startup
│
└── 🧩 CLI Menu
     ├─ 1. Lookup by Investor
     ├─ 2. Lookup by Startup
     ├─ 3. Specific Startup-Investor Pair
     ├─ 4. Show Top Matches
     └─ 5. Exit
```

⚖️ What Needs to Change for True Two-Way Matching

Area	Current	Needed for Two-Way
Elasticsearch Indexing	Only investors are indexed	Also index startups with embeddings
Vector Search	Search investors using startup embedding	Also search startups using investor embedding
Matching Loop	For each startup → top investors	Add: For each investor → top startups
Index Names	`investors` only	Add second: `startups`
Output Format	Startup → Investor matches	Merge both directions intelligently
LLM Justification	Investor-to-startup reasoning	Add alternate prompt wording for reverse direction
CLI Menu	Manual lookup either side	Unified "Explore both ways" mode

---

🧭 Simplified Flow for Two-Way Matching

1. **Create two ES indexes:**
   - `investors` → vector of investor descriptions
   - `startups` → vector of startup descriptions
2. **Index both datasets** with their embeddings.
3. **Run hybrid match twice:**
   - `match_startup_to_investor()` → current flow
   - `match_investor_to_startup()` → new reverse flow

4. **Merge results** intelligently (avoid duplicates, average scores).

5. **Store final combined results** in a JSON file.

Once you understand this flow, the next step is to: ☑ Add `create_index("startups")` ☑ Add `index_startups()` (same logic as investor indexing) ☑ Add reverse search: `search_similar_startups(investor_emb, top_k=5)` ☑ Modify `hybrid_match()` to combine both directions.

Would you like me to now give you the **exact step-by-step code diff plan** (new functions + where to insert them) to make this two-way matching engine work cleanly?